Big Data & Distributed Frameworks

Capstone / Final Project

Real-Time Fleet Monitoring Platform

Apache Kafka – Spark Structured Streaming and Batch – Apache Airflow

Ralph Bou Nader

Master's in Data Engineering – EFREI Paris

Team size: 1 or 2 students. **Total:** 20 points. **Submission:** see Section 8.

---

1 Context

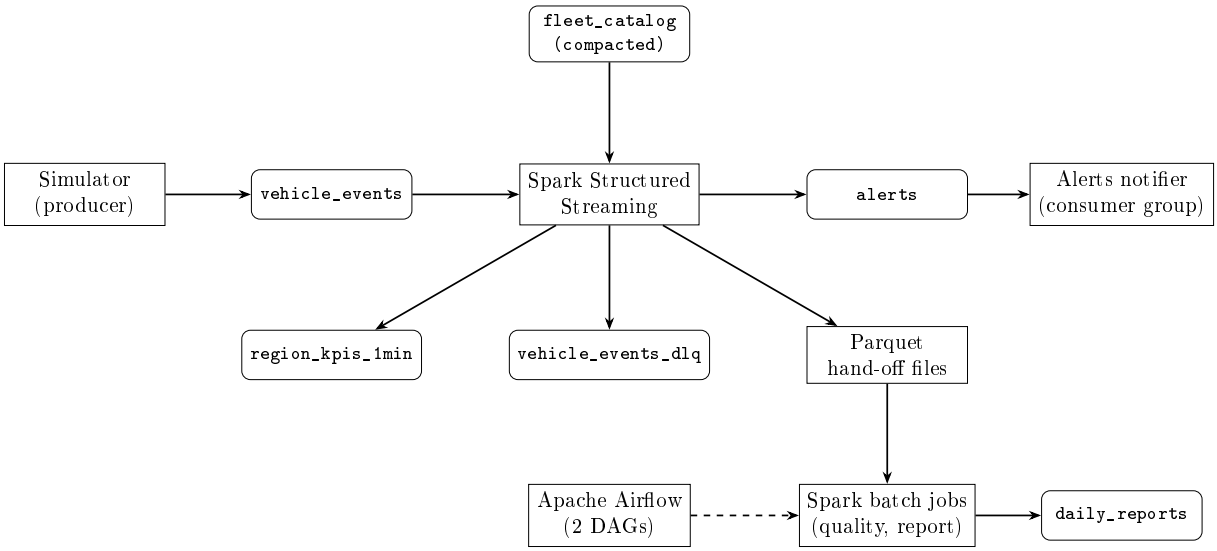
A logistics company operates 40 delivery vehicles across five French regions. Every vehicle emits telemetry (position, speed, fuel level, harsh-braking flag). Operations managers need three things:

- **immediate alerts** when a vehicle speeds or brakes harshly, and when it keeps speeding over several minutes;
- **live regional indicators**, refreshed every minute;
- **a daily report**, produced automatically and only if the day's data passes a quality gate.

The feed is not clean: it contains duplicated messages, late messages, corrupted messages, and one vehicle that sends much more traffic than the others. Your platform must survive all of this and you must *prove* that it does.

The platform uses exactly three technologies: **Kafka** (transport, in and out), **Spark** (streaming and batch processing) and **Airflow** (orchestration and monitoring). Every architectural choice must be justified in your report with the distributed-systems vocabulary of Chapter 1 (replication, consensus, CAP/PACELC) and, where relevant, the resource model of Chapter 2.

2 Architecture



Dashed arrows: orchestration and monitoring by Airflow. The Parquet files are only a hand-off between the streaming job and the daily batch jobs; they are not a storage design you are asked to optimise.

3 Provided material and rules

Provided: a Docker Compose file starting one Kafka broker in KRaft mode, a topic-creation script and the telemetry simulator with its fault injection (rates: 3% anomalies, 1% duplicates, 2% late events of 3–6 minutes, 1% corrupted messages; option `--hot-vehicle` sends about 40% of traffic from one vehicle). The six topics are `vehicle_events` (6 partitions), `fleet_catalog` (3, compacted), `alerts` (3), `region_kpis_1min` (3), `vehicle_events_dlq` (3) and `daily_reports` (1).

Everything else is yours to write. Language: Python (PySpark 3.5, confluent-kafka, Airflow 2.9). Your whole project must start with `docker compose up` and the commands of your README. Code that does not run is graded as missing.

Message format (JSON, key = `vehicle_id`): `event_id`, `vehicle_id`, `event_time` (ISO 8601), `latitude`, `longitude`, `speed_kmh`, `fuel_level_pct`, `harsh_braking`. Catalog record: `vehicle_id`, `plate`, `vehicle_type`, `home_region`, `max_payload_kg`.

Part A – Kafka layer **(4 points)**

1. Publish the catalog to the compacted topic with an idempotent producer. Explain what compaction guarantees and what it does *not* guarantee (when is old data actually removed?).
2. Write the alerts notifier: a consumer in group `alerts-notifier`, manual offset commit *after* the alert is handled, cooperative-sticky assignment. Start two instances and show the partition assignment before and after you kill one.
3. Justify the partition count and the key of each topic. What breaks if `vehicle_events` is keyed randomly instead of by vehicle? What breaks with only one partition?

4. State and justify the delivery semantics of each hop (producer to broker, broker to Spark, Spark to `alerts`, `alerts` to notifier).

Part B – Spark Structured Streaming (6 points)

One Spark application, four streaming queries.

1. **Validation and dead-letter queue.** Parse with an explicit schema. Every invalid message (broken JSON, missing field, unparsable timestamp, speed outside 0–250) goes to `vehicle_events_dlq` with a `reason`. Beware of NULL semantics: a comparison with NULL is neither true nor false, and a naive filter silently loses the message instead of routing it.
2. **Deduplication** on `event_id` with a 2-minute watermark. Explain why the state is bounded and what happens to a duplicate arriving after the watermark.
3. **Enrichment** with the catalog read from Kafka (stream–static join, broadcast). Vehicles unknown to the catalog must get the region `UNKNOWN`, not disappear. What happens to the join when the catalog changes while the job runs? Document the limitation.
4. **Query 1 – per-event** (`foreachBatch`): write the micro-batch as Parquet partitioned by `event_date` and `batch_id`, and publish point alerts (speed above 120 km/h or harsh braking) to `alerts`. The write must be idempotent when a batch is replayed after a crash; explain why the alert publication is not, and what the consequences are.
5. **Query 2 – tumbling window**: per region and per 1-minute event-time window, number of events, mean and maximum speed, number of speeding events, published to `region_kpis_1min` in update mode.
6. **Query 3 – sliding window**: 5-minute windows sliding every minute, per vehicle; a vehicle whose mean speed exceeds 100 km/h over at least 5 events raises a `sustained_speeding` alert.
7. Explain the role of each checkpoint directory and what happens if you delete one while keeping the others.

Part C – Spark batch (3 points)

1. **Quality job**: for one date, compute the number of rows, duplicated `event_id`s and percentage of rows with region `UNKNOWN`; write a JSON verdict (`passed` true or false, with reasons). Thresholds are yours; justify them.
2. **Daily report job**: per vehicle and per region (events, mean speed, maximum speed, speeding events, harsh-braking events) and the five worst vehicles. Show with the Spark UI (or `explain()`) that only the partitions of the requested date are read.
3. **Skew**: run the simulator with `--hot-vehicle`. Show the skew in the Spark UI (task-time distribution of the aggregation stage). Implement salting (two-stage aggregation) and show the improvement. Compare with Adaptive Query Execution alone.

Part D – Airflow orchestration (4 points)

1. **DAG** `fleet_daily_pipeline` (daily): a sensor in *reschedule* mode waits for the day's files; a task checks that the six topics exist (retries with delay); the quality job runs; a *branch* task reads the verdict. If it passed: report job, then publication of the report to `daily_reports`. If it failed: publish a `data_quality_failed` alert instead. A final task joins both paths and must succeed whichever path was taken (check the trigger rule).
2. **DAG** `kafka_lag_monitor` (every 5 minutes): measure the lag of `alerts-notifier` per partition; if it exceeds a threshold, publish a `consumer_lag_high` alert.
3. Explain the difference between the logical date and the execution time, why `catchup` is set the way you chose, and what a backfill of the last 7 days would do in your pipeline. Are your tasks idempotent? Prove it by running one twice.
4. Explain why a sensor in *poke* mode would be a poor choice here.

Part E – Experiments and report **(3 points)**

Failure experiments. For each one, state your hypothesis *before* running it, then give observations and explanation.

1. Kill the streaming job with `kill -9` and restart it. Count Parquet rows and alerts: what is duplicated, what is lost, what is intact?
2. Stop the alerts notifier for 10 minutes, then restart it. Observe the lag, the monitor DAG and the catch-up.
3. Stop the Kafka broker for 1 minute during production. What do the producer, Spark and the notifier do? Which settings control it?
4. Send a burst at ten times the normal rate. Where is the bottleneck? What would you change first?

Performance study. One table, two or more varying parameters among: number of partitions, `maxOffsetsPerTrigger`, trigger interval, `spark.sql.shuffle.partitions`, salting. Measure throughput and end-to-end latency (event time to alert time). Conclude with a configuration you recommend and why.

Report (10 pages maximum, PDF): architecture diagram; the justifications asked above; a section "Distributed-systems analysis" linking your choices to Chapter 1 (where is the consensus? which side of CAP does each component take?); the experiments; limits of your solution and what you would do with one more month.

4 Grading

Part	Points	Main criteria
A – Kafka	4	correct keys/partitions, idempotent producer, manual commits, justified semantics
B – Streaming	6	DLQ without loss, bounded dedup, three windows/-queries correct, idempotent batch write
C – Batch	3	partition pruning shown, skew demonstrated and fixed
D – Airflow	4	correct sensor mode, branching, trigger rule, idempotence, lag DAG
E – Experiments, report	3	hypotheses before results, measurements, honest limits

Deductions: project does not start from the README (up to −4); unexplained copied code (up to −5); missing individual contribution statement for a team of two (−2). An oral defence of 10 minutes may adjust the mark of each member individually.

5 Suggested timeline

Week 1	Kafka layer, simulator running, notifier with two instances
Week 2	Streaming job: validation, DLQ, dedup, enrichment, Query 1
Week 3	Queries 2 and 3, batch jobs, skew study
Week 4	Airflow DAGs, failure experiments, performance study
Week 5	Report, README, rehearsal of the defence

6 Hints

- The Kafka source for Spark needs the package `spark-sql-kafka-0-10_2.12:3.5.1` (group `org.apache.spark`).
- Kafka sink: columns named `key` and `value` (string or binary).
- `dropDuplicatesWithinWatermark` is available from Spark 3.5.
- The Spark option `partitionOverwriteMode` (value `dynamic`) changes what `overwrite` does to a partitioned write.
- A consumer that only reads committed offsets does not need to `subscribe`.
- Airflow: a `PythonSensor` in reschedule mode, the `@task.branch` decorator, and the trigger rule named `none_failed_min_one_success`.

7 Rules

Teams of 1 or 2; no collaboration between teams. External libraries beyond those listed need written approval. You may use documentation freely; any other source of code must be cited in the report.

8 Submission

A `.zip` named `NAME1_NAME2_capstone.zip` containing: the full project with README, the PDF report, the raw outputs of your experiments (logs, screenshots of the Spark UI and Airflow UI), and a one-paragraph contribution statement per student. Deadline: announced on Moodle.